zeroriscy_decoder

OVERVIEW

The Main Instruction Decoder acts as the central control unit of the Zero-Riscy processor. Its primary responsibility is to decode incoming 32-bit instructions and generate all the necessary control signals required for proper instruction execution.
This module serves as the decision-making block of the processor, determining how each instruction should be processed by the ALU, register file, memory unit, branch unit, and CSR block.
It analyzes the instruction format, identifies the instruction type, and activates the correct execution path by generating control signals for arithmetic operations, memory access, branching, jumps, and exception handling.

FUNCTIONAL DESCRIPTION: CORE RESPONSIBILITY

The main purpose of the decoder is to interpret the fetched instruction and control the complete execution flow of the processor.
It performs the following tasks:

- Decodes instruction opcode and format
- Selects ALU operation type and input operands
- Controls register file read/write operations
- Generates memory access signals
- Handles branch and jump instructions
- Detects illegal instructions
- Controls CSR access operations
- Supports exception handling for faults and invalid instructions

This makes the decoder one of the most important modules in the processor pipeline.

INPUT SIGNALS

- instr_rdata_i [31:0] -> Instruction input fetched from instruction memory. This is the main instruction that the decoder analyzes and interprets.
- branch_mux_i -> Branch control feedback signal used to determine whether a branch instruction should be taken.
- jump_mux_i -> Jump control feedback signal used for jump instruction execution and control flow updates.
- data_misaligned_i -> Memory alignment error indicator. Active when memory access is attempted on an invalid or misaligned address.
- illegal_c_insn_i -> Illegal compressed instruction flag received from the compressed instruction decoder.
- deassert_we_i -> Pipeline stall control signal used to temporarily disable write operations during hazards or pipeline control conditions.

OUTPUT SIGNALS: ALU & REGISTER FILE CONTROL

ALU Control Signals:

- alu_operator_o -> Selects which operation the ALU must perform (e.g., ADD, SUB, AND, OR, XOR, Shift, Comparison).
- alu_op_a_mux_sel_o -> Selects the source for ALU Operand A.
- alu_op_b_mux_sel_o -> Selects the source for ALU Operand B.

These signals ensure the ALU receives the correct operands for execution.

Register File Control:

- regfile_we_o -> Register write enable signal. Determines whether the result should be written back into the register file.

OUTPUT SIGNALS: MEMORY, FLOW, EXCEPTIONS, & CSR

Memory Control Signals:

- data_req_o -> Memory access request signal used for load/store instructions.
- data_we_o -> Data memory write enable signal (0 -> Read operation, 1 -> Write operation).

Control Flow Signals:

- branch_in_id_o -> Indicates branch instruction in decode stage.
- jump_in_id_o -> Indicates jump instruction in decode stage.

Exception and Error Signals:

- illegal_insn_o -> Illegal instruction detection flag. Raised when the instruction does not match any valid supported instruction.

CSR Control Signals:

- csr_access_o -> Enables CSR access operations.
- csr_op_o -> Selects CSR operation type (e.g., Read, Write, Set bits, Clear bits).

WORKING PRINCIPLE: STEPS 1 & 2

Step 1: Opcode Extraction

The decoder first examines the opcode field of the instruction (instr_rdata_i[6:0]). This field identifies the instruction category. Examples include arithmetic, load/store, branch, jump, CSR, and system instructions. Opcode detection is the first and most important step in decoding.

Step 2: Instruction Type Identification

Based on the opcode, the decoder determines the specific instruction type:

- R-type instruction
- I-type instruction
- Load/Store instruction
- Branch / Jump instruction
- CSR/System instruction

Each instruction type requires a different execution path and control logic.

WORKING PRINCIPLE: STEP 3 - CONTROL SIGNAL GENERATION

After identifying the instruction type, the decoder generates the required control signals:

- Arithmetic Instructions: Select ALU operation and enable register write-back.
- Load Instructions: Enable memory request, select address calculation through ALU, and enable register write-back.
- Store Instructions: Enable memory request, enable memory write, and disable register write-back.
- Branch and Jump Instructions: Enable control flow signals, select branch/jump path, and update Program Counter.
- CSR Instructions: Enable CSR access and select appropriate CSR operation.

This stage ensures correct execution of every instruction.

WORKING PRINCIPLE: STEP 4 - EXCEPTION HANDLING

The decoder also checks for execution faults such as:

- Illegal instructions
- Invalid compressed instructions
- Misaligned memory accesses

If any issue is detected, the corresponding exception signal is raised (e.g., illegal_insn_o = 1). This protects the processor from unsafe or unsupported operations.

INTERNAL DESIGN APPROACH

Combinational Logic Based:
The decoder is primarily implemented using combinational logic. This means no clock dependency, immediate instruction decoding, and fast control signal generation. This improves pipeline speed and reduces decode latency.

Centralized Control Logic:
Since all major execution decisions originate from this module, it acts as the processor's main control center. It coordinates the ALU, Register File, LSU, Branch Unit, and CSR Block. This centralized architecture improves design clarity and maintainability.

Exception Integration:
The decoder also integrates exception detection with normal instruction flow. This ensures safe execution, reliable trap generation, and proper system recovery without requiring separate heavy exception logic.

KEY ADVANTAGES & SUMMARY

Key Design Advantages:

- Fast instruction decoding using combinational logic
- Centralized control generation for all execution units
- Efficient ALU and memory operation control
- Reliable branch and jump handling
- Built-in illegal instruction detection and CSR support

- Safe handling of misaligned memory accesses and exceptions

SUMMARY:
The zeroriscy_decoder module is the heart of processor control in the Zero-Riscy core. It decodes 32-bit instructions and generates all execution control signals for the ALU, memory, register file, and branching. It supports CSR operations, detects illegal instructions, and ensures smooth and correct processor execution. Without this module, the processor would not be able to understand or execute instructions correctly, making it one of the most essential blocks in the complete processor architecture.